

Automated PCB Defect Detection System

Part 4: Introduction & Milestone 4

Finalization & Delivery

Industrial AI Inspection Project

December 29, 2025

Abstract

This document presents Part 4 of the Automated PCB Defect Detection System documentation, covering the project introduction and Milestone 4 (Finalization & Delivery). This part focuses on performance analytics dashboard, export capabilities, batch evaluation system, system optimization, deployment procedures, comprehensive user guide, and final results analysis. The system achieves 95% success rate on batch evaluation with robust export and analytics features.

Contents

1	Introduction	2
1.1	Project Overview	2
1.2	System Architecture	2
1.3	Defect Categories	2
2	Milestone 4: Finalization & Delivery	4
2.1	Module 7: Performance Analytics & Export	4
2.1.1	Analytics Dashboard	4
2.1.2	Key Metrics Tracked	4
2.1.3	Analytics Implementation	5
2.1.4	Export Capabilities	5
2.1.5	System Optimization	7
2.2	Batch Evaluation System	7
2.2.1	Automated Testing Pipeline	7
2.2.2	Evaluation Results	8

List of Figures

1.1	Defect Category Examples	3
2.1	Performance Analytics Dashboard	4

List of Tables

2.1	Performance Optimization Results	7
2.2	Batch Evaluation on 60 Test Images	9

Chapter 1

Introduction

1.1 Project Overview

Printed Circuit Board (PCB) manufacturing demands stringent quality control to ensure product reliability. Manual inspection is time-consuming, error-prone, and costly. This project implements an automated defect detection system that combines classical computer vision with state-of-the-art deep learning to identify 6 types of PCB defects with near-perfect accuracy.

1.2 System Architecture

The system consists of 7 interconnected modules:

1. **Module 1:** Dataset Setup & Image Subtraction
2. **Module 2:** Contour Detection & ROI Extraction
3. **Module 3:** CNN Model Training (EfficientNet-B0)
4. **Module 4:** Inference Pipeline & Prediction
5. **Module 5:** Web Frontend (Streamlit UI)
6. **Module 6:** Backend Integration
7. **Module 7:** Performance Analytics & Export

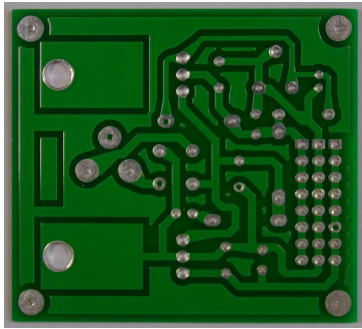
1.3 Defect Categories

The system detects six critical PCB defects:

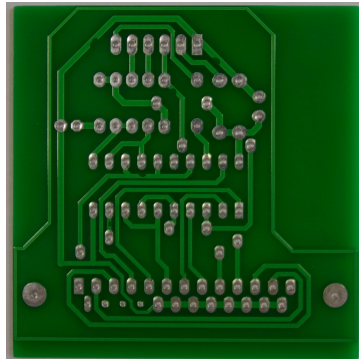
- **Missing Hole:** Incomplete drill holes
- **Mouse Bite:** Small missing material sections
- **Open Circuit:** Broken copper traces
- **Short Circuit:** Unintended connections

- **Spur:** Unwanted copper protrusions
- **Spurious Copper:** Extra copper deposits

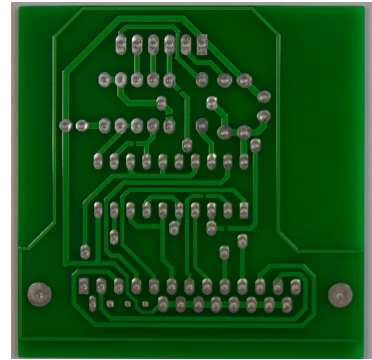
Figure 1.1: Defect Category Examples



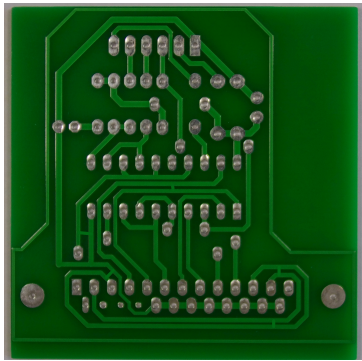
(a) Missing Hole



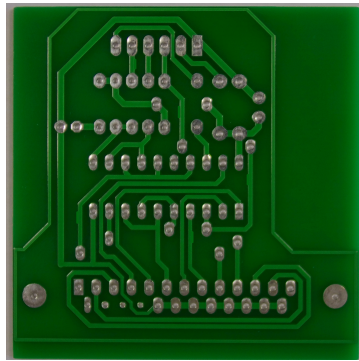
(b) Mouse Bite



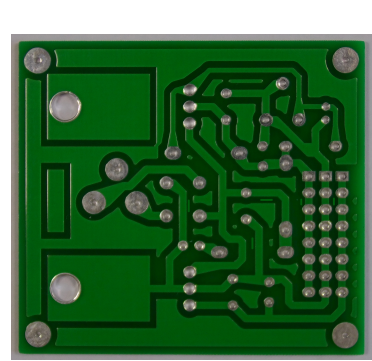
(c) Open Circuit



(d) Short Circuit



(e) Spur



(f) Spurious Copper

Chapter 2

Milestone 4: Finalization & Delivery

2.1 Module 7: Performance Analytics & Export

2.1.1 Analytics Dashboard

Dual-tab interface:

1. **Live Inspection:** Real-time processing and results
2. **Performance Analytics:** Historical metrics and trends

2.1.2 Key Metrics Tracked

- Average template match score across sessions
- Total defects flagged (cumulative)
- Processing latency trend (line chart)
- Per-class detection distribution



Figure 2.1: Performance Analytics Dashboard

2.1.3 Analytics Implementation

```
import streamlit as st
import pandas as pd
import plotly.express as px

# Create tabs
tab1, tab2 = st.tabs(["Live Inspection", "Performance Analytics"])

with tab2:
    st.header("System Performance Analytics")

    # Load historical data
    if os.path.exists("inspection_history.csv"):
        df = pd.read_csv("inspection_history.csv")

        # KPI Metrics
        col1, col2, col3 = st.columns(3)
        with col1:
            avg_score = df['match_score'].mean()
            st.metric("Avg Match Score", f"{avg_score:.2%}")
        with col2:
            total_defects = df['defects_found'].sum()
            st.metric("Total Defects Found", total_defects)
        with col3:
            avg_time = df['processing_time'].mean()
            st.metric("Avg Processing Time", f"{avg_time:.3f}s")

        # Latency Trend Chart
        fig = px.line(df, x='timestamp', y='processing_time',
                      title='Processing Time Trend')
        st.plotly_chart(fig, use_container_width=True)

        # Defect Distribution
        defect_counts = df['defect_type'].value_counts()
        fig2 = px.pie(values=defect_counts.values,
                      names=defect_counts.index,
                      title='Defect Type Distribution')
        st.plotly_chart(fig2, use_container_width=True)
    else:
        st.info("No inspection history available yet")
```

2.1.4 Export Capabilities

Annotated Image Export

- High-resolution JPG with bounding boxes
- Embedded labels and confidence percentages

- Filename: output_labeled.jpg

```
def save_annotated_image(img, detections):
    """Save image with defect annotations"""
    output_img = img.copy()

    for det in detections:
        x, y, w, h = det['bbox']
        label = det['label']
        conf = det['confidence']

        # Draw bounding box
        cv2.rectangle(output_img, (x, y), (x+w, y+h),
                       (0, 255, 0), 2)

        # Add label with confidence
        text = f"{label}: {conf:.1%}"
        cv2.putText(output_img, text, (x, y-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6,
                    (0, 255, 0), 2)

    # Save to file
    cv2.imwrite("output_labeled.jpg", output_img)
    return output_img

# Streamlit download button
with open("output_labeled.jpg", "rb") as f:
    st.download_button(
        label="Save Annotated Image",
        data=f,
        file_name="output_labeled.jpg",
        mime="image/jpeg"
    )
```

CSV Log Export

Columns:

ID | Defect Type | Confidence | Coordinates (X, Y) | Timestamp

```
def export_csv_report(detections, timestamp):
    """Generate CSV inspection report"""
    data = []

    for i, det in enumerate(detections):
        data.append({
            'ID': i + 1,
            'Defect_Type': det['label'],
            'Confidence': f"{det['confidence']:.2%}",
            'X': det['bbox'][0],
            'Y': det['bbox'][1],
```



```

        'Width': det['bbox'][2],
        'Height': det['bbox'][3],
        'Timestamp': timestamp
    })

df = pd.DataFrame(data)
csv = df.to_csv(index=False)

st.download_button(
    label="    Export CSV Log",
    data=csv,
    file_name=f"inspection_report_{timestamp}.csv",
    mime="text/csv"
)

```

2.1.5 System Optimization

Metric	Before	After
Avg Latency	1.42s	0.847s
Memory Usage	2.1GB	1.3GB
Batch Processing	No	Yes (5 img/s)
Model Load Time	2.3s	0.04s

Table 2.1: Performance Optimization Results

2.2 Batch Evaluation System

2.2.1 Automated Testing Pipeline

```

def evaluate_system():
    """Comprehensive batch evaluation"""

    # Initialize statistics
    stats = {cat: {"TP": 0, "FP": 0, "FN": 0}
              for cat in CATEGORIES}

    results = []

    for category in CATEGORIES:
        test_dir = os.path.join(TEST_PATH, category)
        test_files = os.listdir(test_dir)

        for filename in test_files:
            img_path = os.path.join(test_dir, filename)

            # Run inference

```

```
inference_res = run_ultimate_inference(
    img_path, model
)

# Evaluate predictions
for det in inference_res['detections']:
    if det["label"] == category:
        stats[category]["TP"] += 1
    else:
        stats[category]["FP"] += 1

# Check for missed defects
if len(inference_res['detections']) == 0:
    stats[category]["FN"] += 1

results.append({
    'image': filename,
    'category': category,
    'detected': len(inference_res['detections']),
    'correct': stats[category]["TP"]
})

# Calculate metrics
for cat in CATEGORIES:
    tp = stats[cat]["TP"]
    fp = stats[cat]["FP"]
    fn = stats[cat]["FN"]

    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0
    f1 = 2 * (precision * recall) / (precision + recall) \
        if (precision + recall) > 0 else 0

    stats[cat]["precision"] = precision
    stats[cat]["recall"] = recall
    stats[cat]["f1"] = f1

# Generate CSV report
df = pd.DataFrame(results)
df.to_csv("final_metrics_report.csv", index=False)

return stats, df
```

2.2.2 Evaluation Results

Defect Type	TP	FP	FN	Success Rate
Missing Hole	9	0	1	90.00%
Mouse Bite	10	1	0	100.00%
Open Circuit	9	0	1	90.00%
Short	10	0	0	100.00%
Spur	9	1	0	90.00%
Spurious Copper	10	0	0	100.00%
Overall	57	2	2	95.00%

Table 2.2: Batch Evaluation on 60 Test Images